

CS 546

Computer System Security

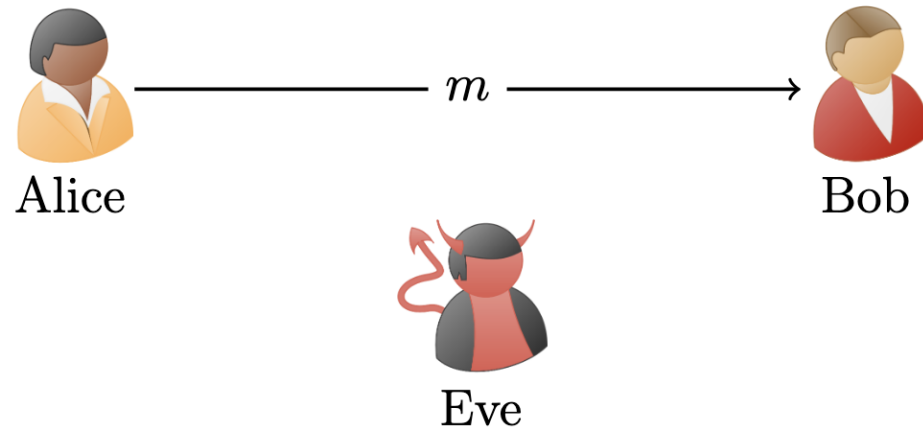
Lecture 3.1: Crypto (Confidentiality - III)

Instructor: Aarushi Goel

Fall 2026

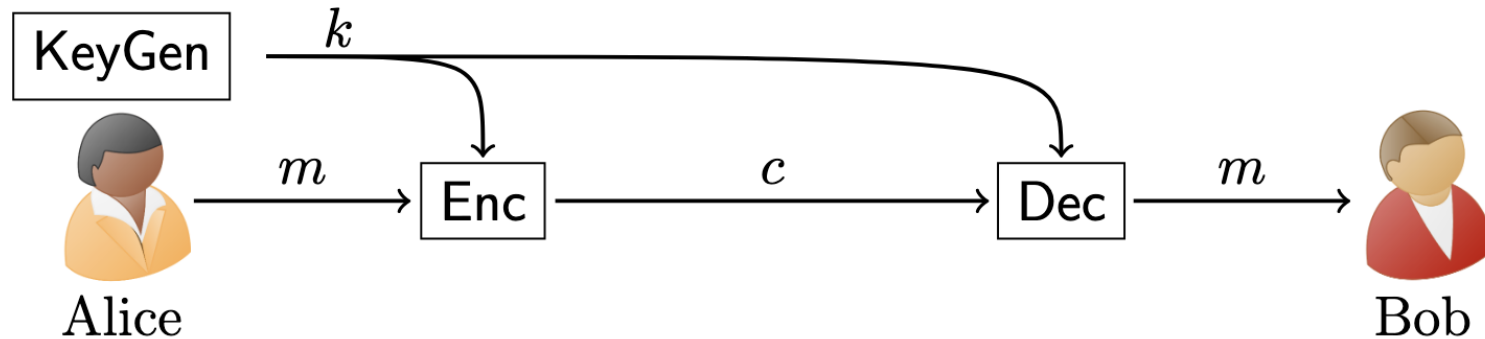
Recap (Secret-Key Encryption)

Problem Setting: Secret Communication



- Alice wants to send a message m to Bob.
- How can Alice convey this message to Bob, while keeping it hidden from an eavesdropper Eve?
- In other words, how can we ensure confidentiality of m ?

Problem Setting: Secret Communication



- **Key Generation:** KeyGen is the randomized process of choosing a secret key k , which is used as input to both Enc and Dec algorithms.
- **Encryption:** Alice transforms (encrypts) the message m – also called the “plaintext” – into a “ciphertext” c .
- **Communication:** Alice sends c to Bob.
- **Decryption:** Bob recovers (decrypts) the original m from c .

Perfect Secrecy

Typical goal in Cryptography:

PRESERVE SECRECY !!

Intuitively speaking, this is what we want:

What Eve learns about m after seeing c , is the same as what they already knew about m .

Formalizing Perfect Secrecy

Setting 1: What did Eve already know about the message?

→ probability distribution over m

→ i.e., $\forall m, \Pr[\text{msg} = m]$

Setting 2: What does Eve learn after seeing c ?

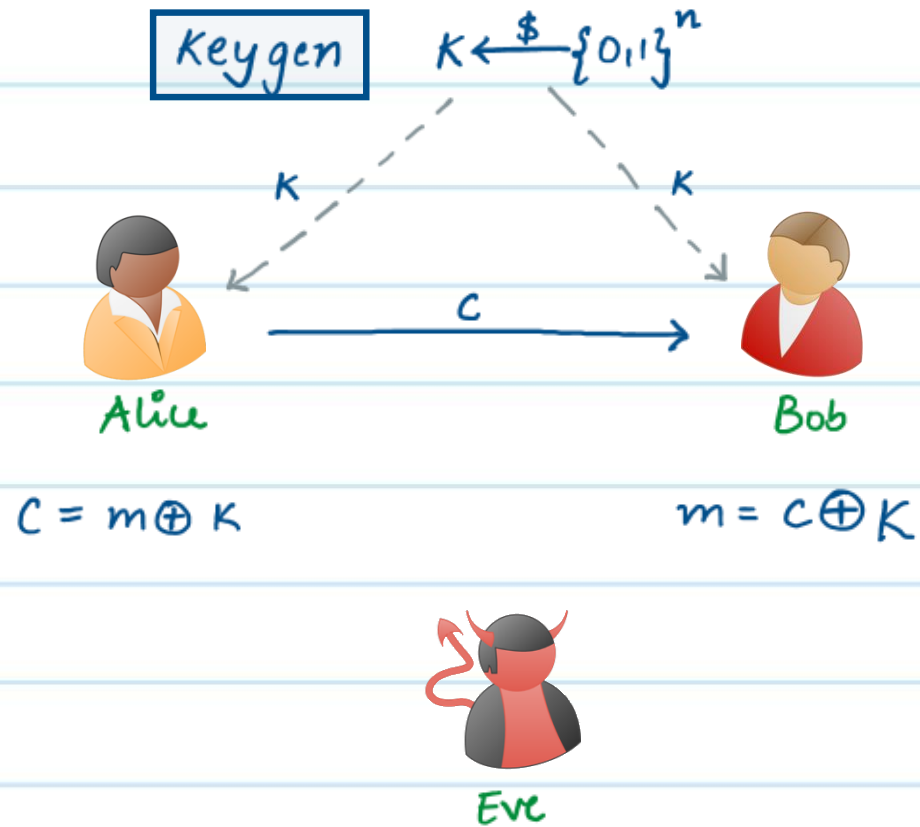
→ New distribution $\Pr[\text{msg} = m \mid \text{view} = c]$

What do we want for secrecy?

Eve's knowledge in setting 1
=

Eve's knowledge in setting 2

One-Time Pad Encryption



Nice Feature :

- * Perfectly secure, i.e., secure against all types of Eve, not just PPT eves

Limitations:

- * Key must be as long as the message
- * Each key can only be used once.

Relaxing Security Requirements

What if the ciphertext is not exactly independent of the message?

Next Best Thing:

The distribution of ciphertexts is close to a distribution that is independent of the message.

Statistical Closeness
Computational Closeness



Computational Security vs Perfect Security

- * For perfect secrecy, we want security against *every* Eve/Adversary.
- * However, this may be an overkill.
- * As we discussed earlier, there are also limitations of perfectly secure encryption schemes.
- * In practice, it might be sufficient to security against computationally feasible attacks as opposed to all possible attacks

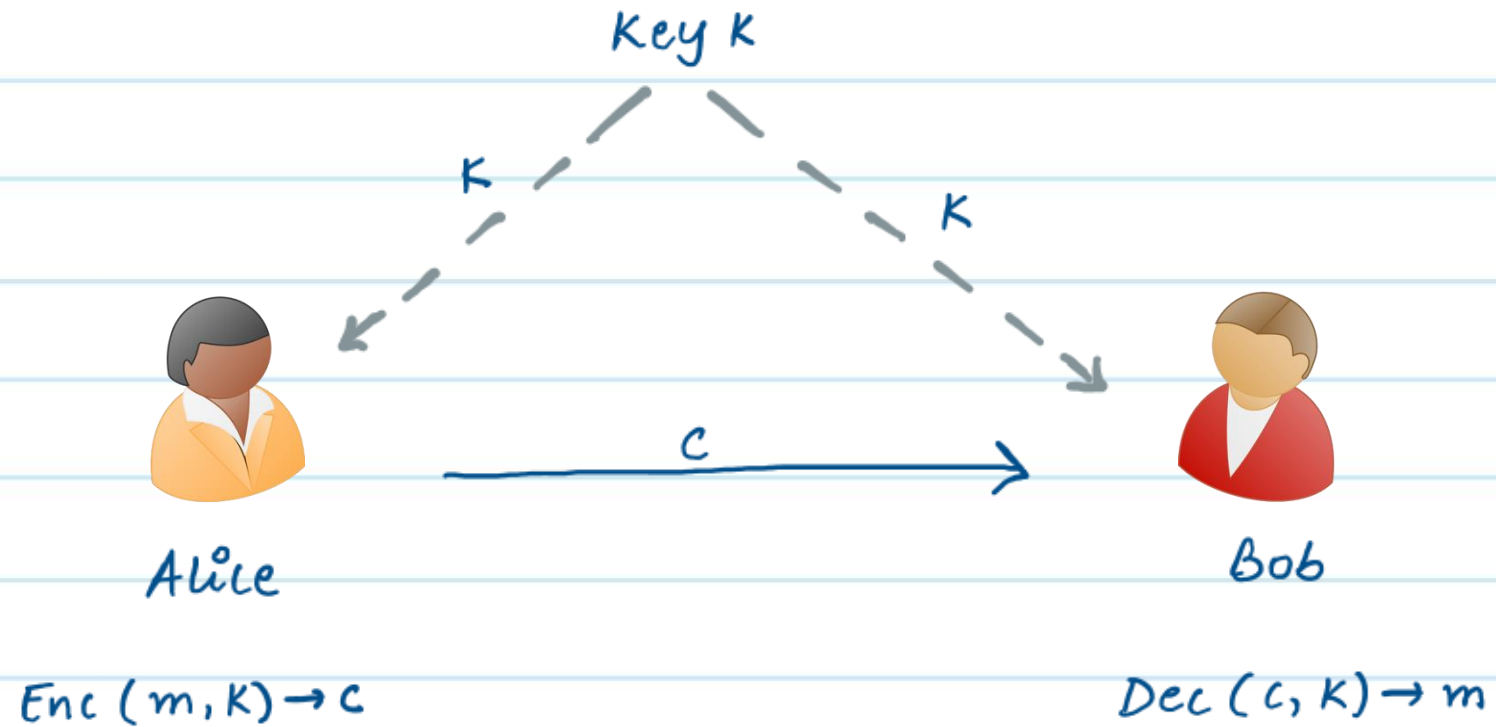


John Nash

“ It doesn't really matter whether attacks are impossible, only whether attacks are computationally infeasible ”

* Modern cryptography is based on this principle *

Computational Security



Computationally bounded
(polynomial-time algorithm)

Computationally Secure Encryption.

An encryption scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is computationally secure if it satisfies **correctness** (as defined previously) and if for every $m_1, m_2 \in \mathcal{M}$,

$$\text{Enc}(K, m_1) \approx_c \text{Enc}(K, m_2)$$

←
Computationally indistinguishable:

- * Hard to distinguish for a computationally bounded adversary.
- * In other words, even if the adversary knows both m_1 & m_2 encryptions of m_1 & m_2 look similar to them if they are computationally bounded.

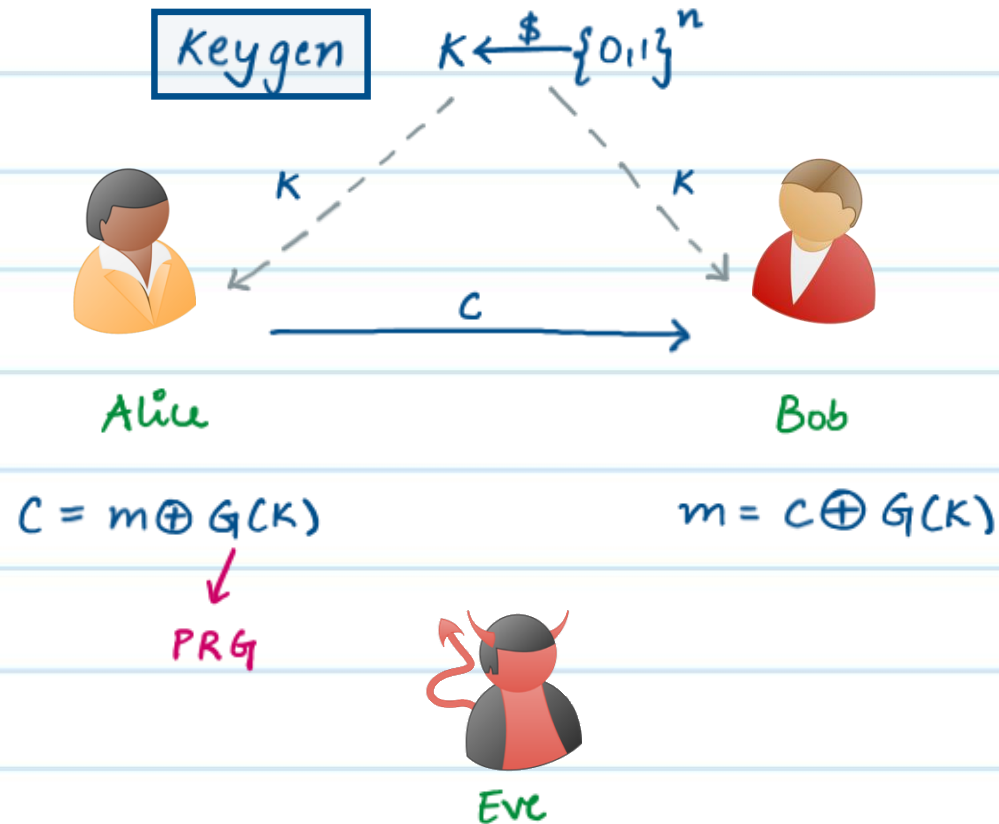
Pseudorandom Generators (PRG)

- * $G : \{0,1\}^n \rightarrow \{0,1\}^{\ell(n)}$, $\ell(n) > n$. PRGs are length expanding.
- * PRGs are deterministic functions
- * The output of a PRG is pseudorandom, i.e., it looks like a randomly sampled string to a computationally bounded adversary.

$$\{r \mid r \xleftarrow{\$} \{0,1\}^{\ell(n)}\} \approx_c \{G(s) \mid s \xleftarrow{\$} \{0,1\}^n\}$$

Pseudorandom OTP Encryption Scheme

* Recall the computationally secure encryption scheme based on PRGs.



Nice Feature:

* Short Key

Limitations:

* Each key can only be used once.

Why?

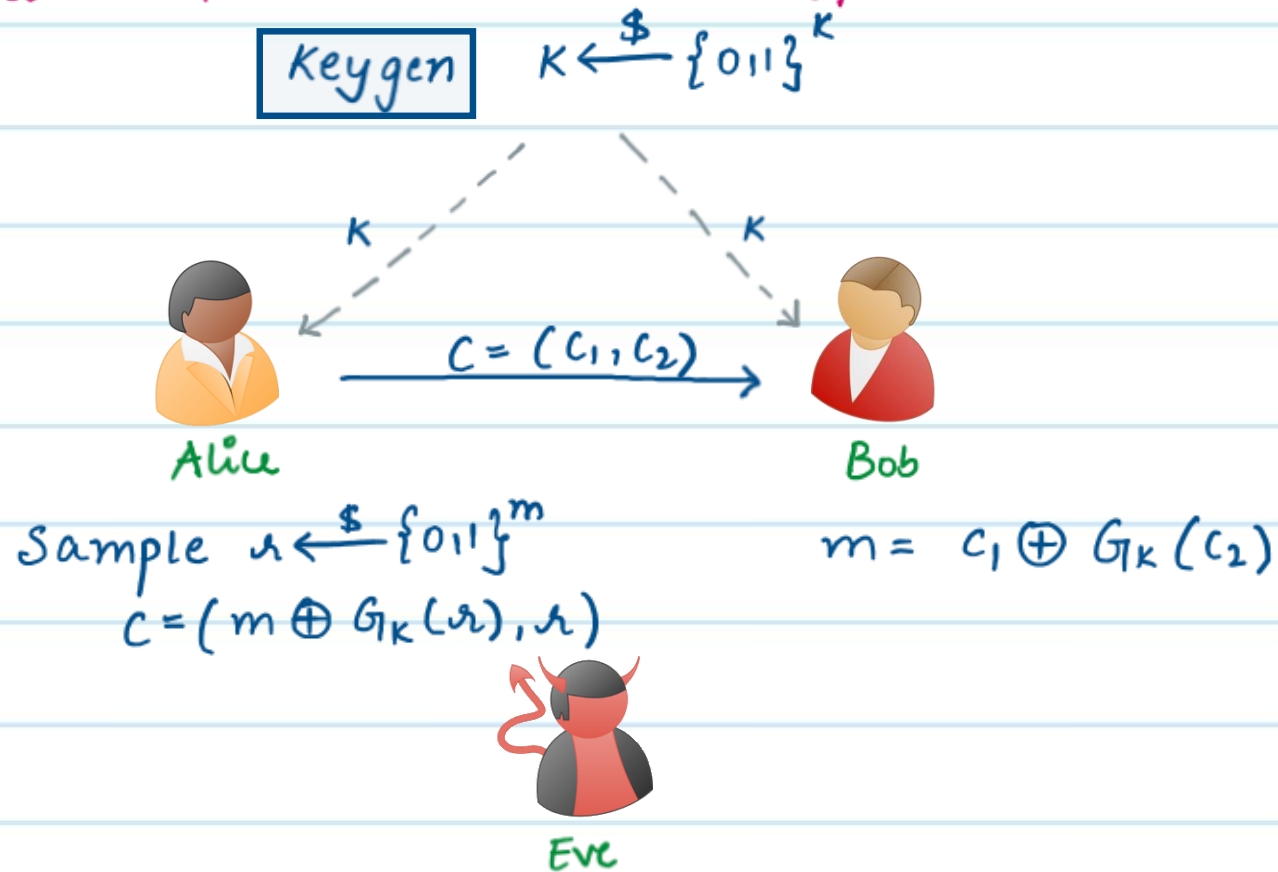
Pseudorandom Functions

- * Pseudorandom functions (PRFs) *look like* a random function and can be described using polynomial number of bits.
- * What does *look like* mean?
 - Look like random functions to a computationally bounded (i.e., PPT) adversary.
 - In other words, they are computationally indistinguishable from random functions.
- * In other words, input-output pairs of a random function & those of a pseudorandom function look computationally indistinguishable.

Encryption using PRFs

* Consider the following encryption scheme.

Such computationally secure encryption where the key can be re-used are also called CPA-secure encryption.



How to Build PRFs?

Two Approaches

Theoretical

- * $PRG \Rightarrow PRF$
- * PRGs can be constructed using well-studied computational hardness assumptions.
- * Provably secure but far from practical.

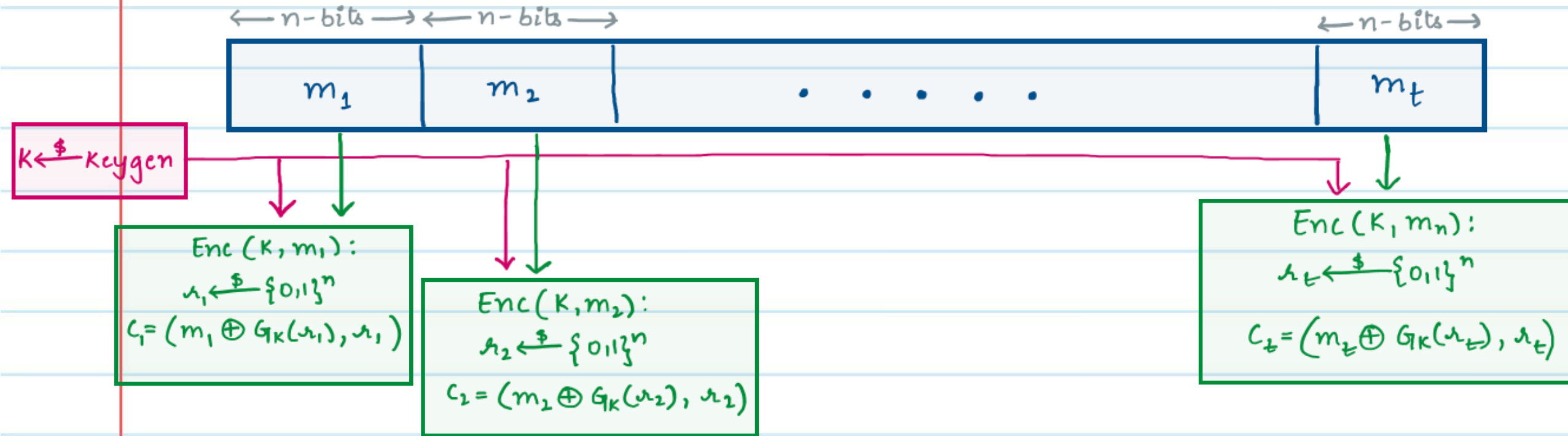
Practical Constructions

- * Start from some design framework ("Appropriately chosen" functions, composed "appropriately many times" look random)
- * Build a candidate construction
- * Do extensive cryptanalysis.
- * Practical but not provably secure (eg. AES, DES etc.)

Modes of Operation

CPA-Secure Encryption for Arbitrary Length Messages

- * Let's start with a PRF $G_K: \{0,1\}^m \rightarrow \{0,1\}^n$. This PRF yields a CPA-secure encryption for encrypting n -bit messages.
- * What if we want to encrypt a tn -bit message?



(Theoretical Construction)

No. of message blocks = t
 size of each message block = n

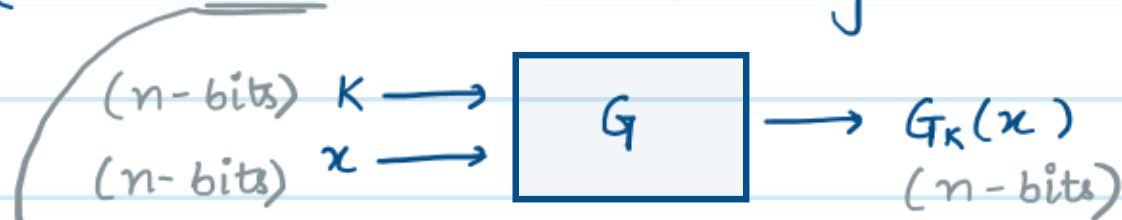
How Good Is This?

	Theoretical Construction				Ideal Goal
Randomness Used	tn -bits				n -bits
Ciphertext Size	$2tn$ -bits				$tn+n$ -bits
Parallelizable ciphertext computation	yes				yes
Reusable Randomness	No				yes
Minimal Assumption	PRF				PRF
CPA Secure?	yes				yes

Block-cipher Modes of Operation

Practical ways of encrypting long messages.

- * Given: Let $\{G_k\}$ be a length-preserving PRF
(or a PRP) with block-length n



pseudorandom permutation is function that is indistinguishable from a random permutation.

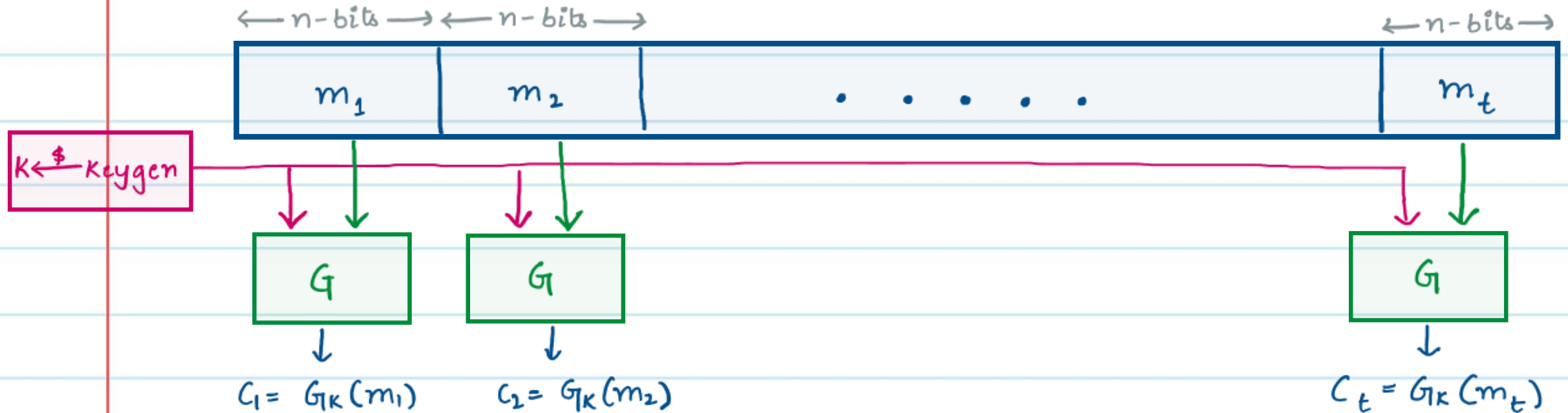
we will assume that G_k^{-1} is also known.

- * Goal: We want to encrypt $m = m_1, \dots, m_t$ using G_k with ciphertext length as small as possible and as less randomness as possible.

What can we do if the message length is not a multiple of n ?

Electronic Code Book (ECB) Mode

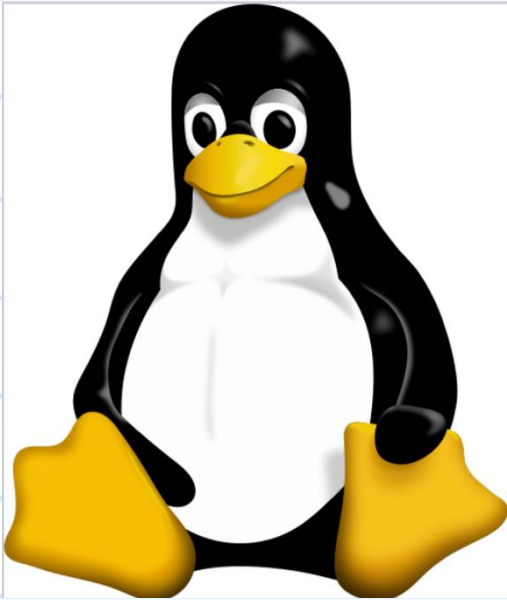
Let $\{G_k\}$ be a family of PRPs.



* Decryption: $\forall i \in [t]$, compute $m_i = G_K^{-1}(c_i)$.

Is this CPA-secure? Why NOT?

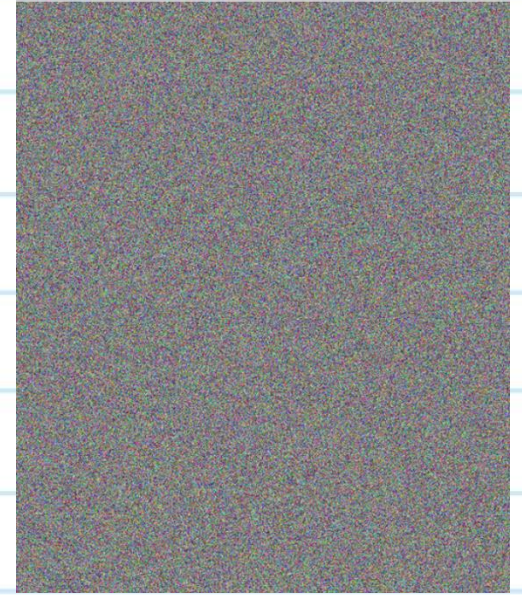
Visible Insecurity of ECB



Plain Image



ECB Encrypted



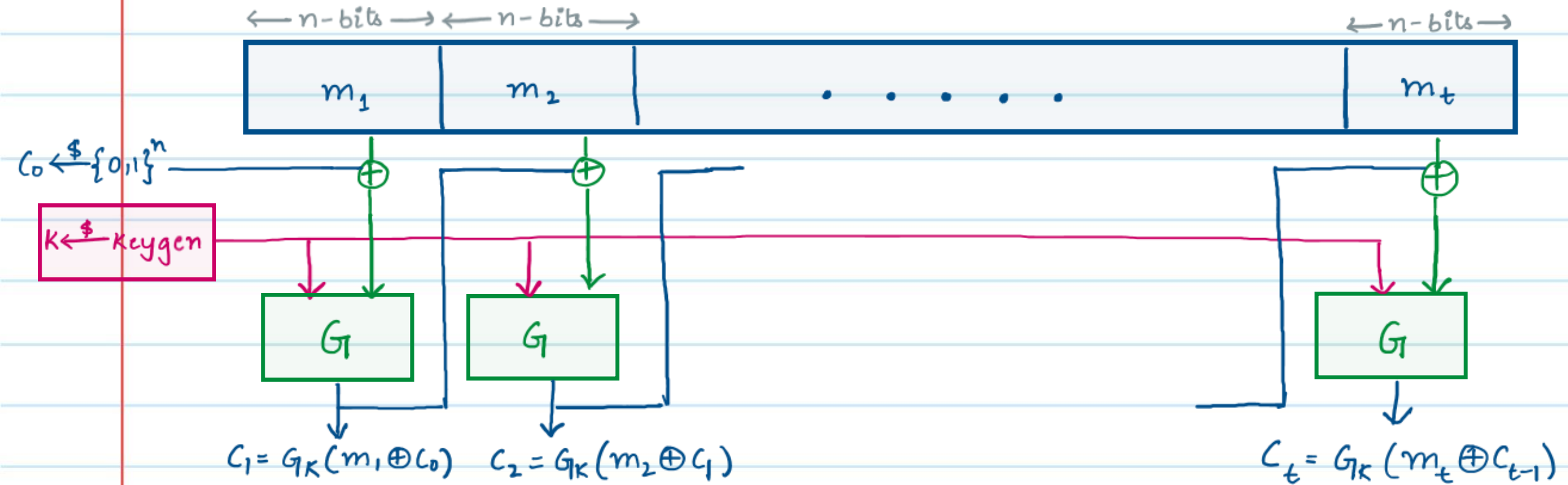
Securely Encrypted

How Good Is This?

	Theoretical Construction	ECB			Ideal Goal
Randomness Used	tn -bits	0 -bits			n -bits
Ciphertext Size	$2tn$ -bits	tn -bits			$tn+n$ -bits
Parallelizable ciphertext computation	yes	yes			yes
Reusable Randomness	No	—			yes
Minimal Assumption	PRF	PRP			PRF
CPA Secure?	Yes	No			Yes

Cipher Block Chaining (CBC) Mode (Used in SSL 3.0 and TLS 1.0)

Let $\{G_K\}$ be a family of PRPs.



* Decryption: $\forall i \in [t]$, compute $m_i = G_K^{-1}(c_i) \oplus c_{i-1}$

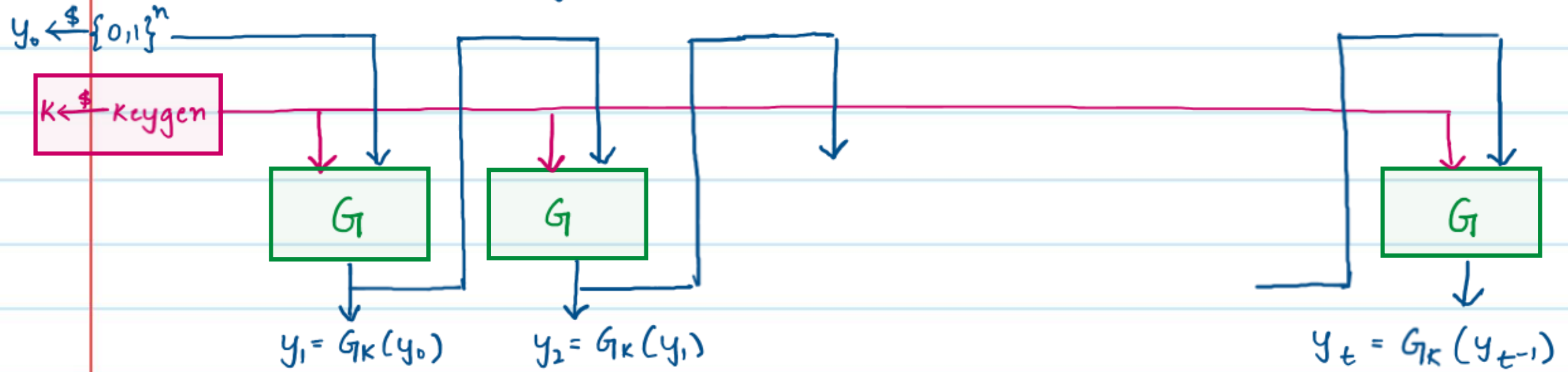
Is this CPA-secure?

How Good Is This?

	Theoretical Construction	ECB	CBC		Ideal Goal
Randomness Used	tn -bits	0 -bits	n -bits		n -bits
Ciphertext Size	$2tn$ -bits	tn -bits	$tn+n$ -bits		$tn+n$ -bits
Parallelizable ciphertext computation	yes	yes	No		yes
Reusable Randomness	No	-	-		yes
Minimal Assumption	PRF	PRP	PRP		PRF
CPA Secure?	Yes	No	yes		Yes

Output Feedback (OFB) Mode

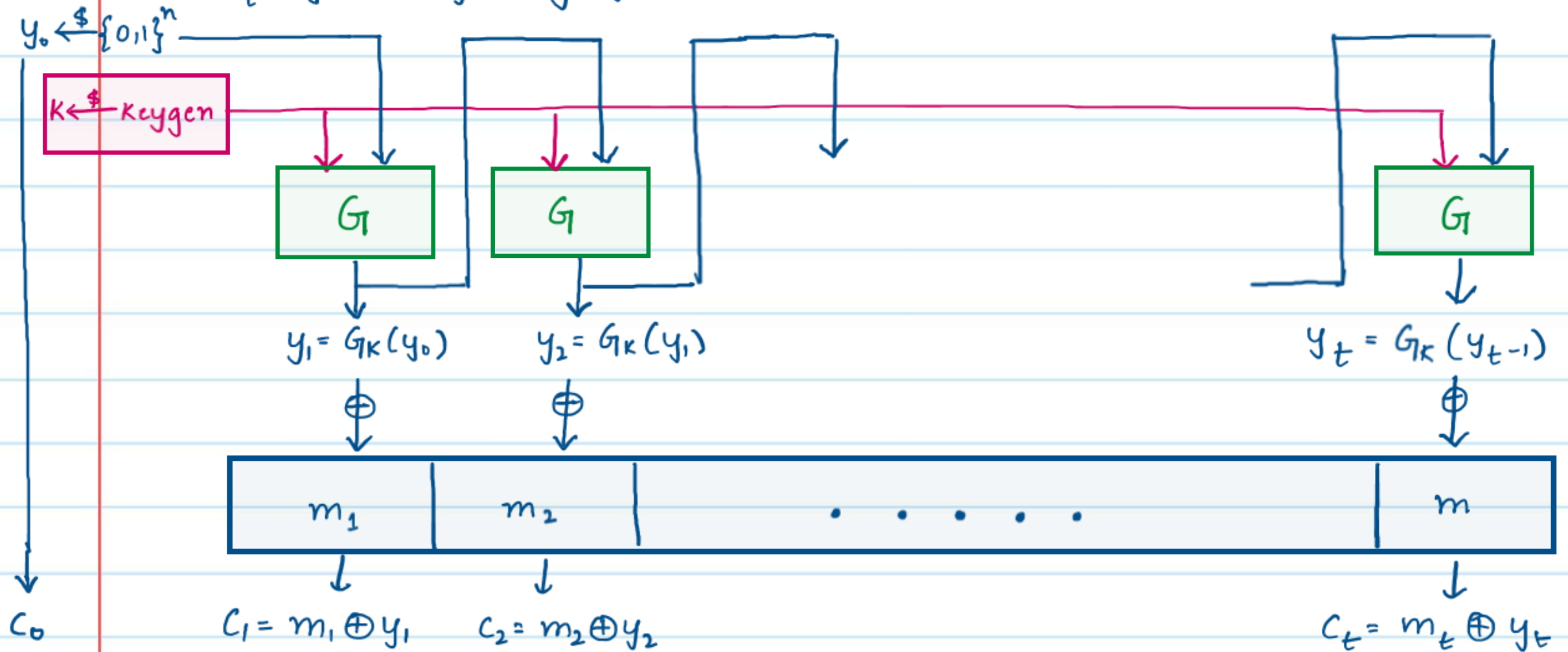
Let $\{G_k\}$ be a family of PRFs



* Generate a pseudorandom stream of one-time pads.

Output Feedback (OFB) Mode (Used in multimedia streaming)

Let $\{G_k\}$ be a family of PRFs.



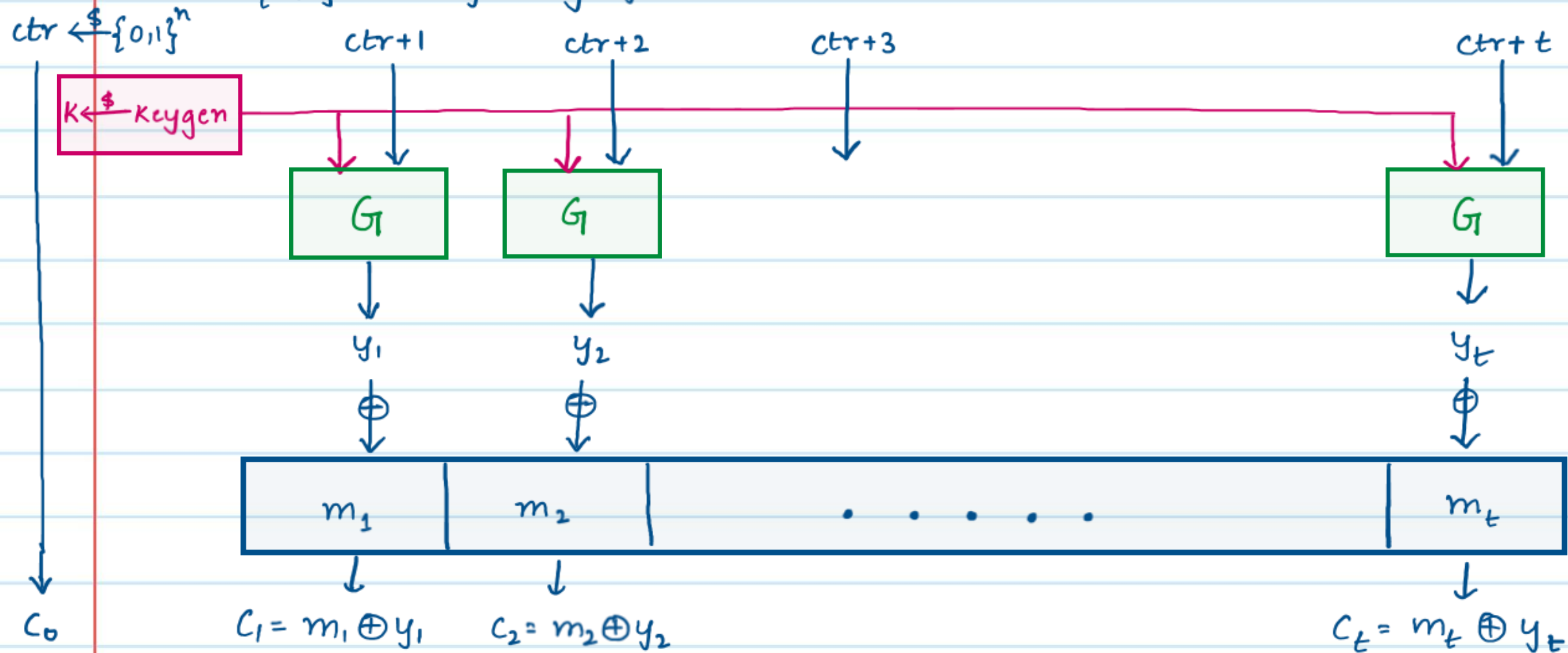
* Use the pseudorandom stream to mask messages

How Good Is This?

	Theoretical Construction	ECB	CBC	OFB	Ideal Goal
Randomness Used	tn -bits	0-bits	n -bits	n -bits	n -bits
Ciphertext Size	$2tn$ -bits	tn -bits	$tn+n$ -bits	$tn+n$ -bits	$tn+n$ -bits
Parallelizable ciphertext computation	yes	yes	No	No, but pre-computable	yes
Reusable Randomness	No	-	-	-	yes
Minimal Assumption	PRF	PRP	PRP	PRF	PRF
CPA Secure?	Yes	No	yes	yes	yes

Counter (CTR) Mode (Used in VPNs, cloud computing etc.)

Let $\{G_k\}$ be a family of PRFs.



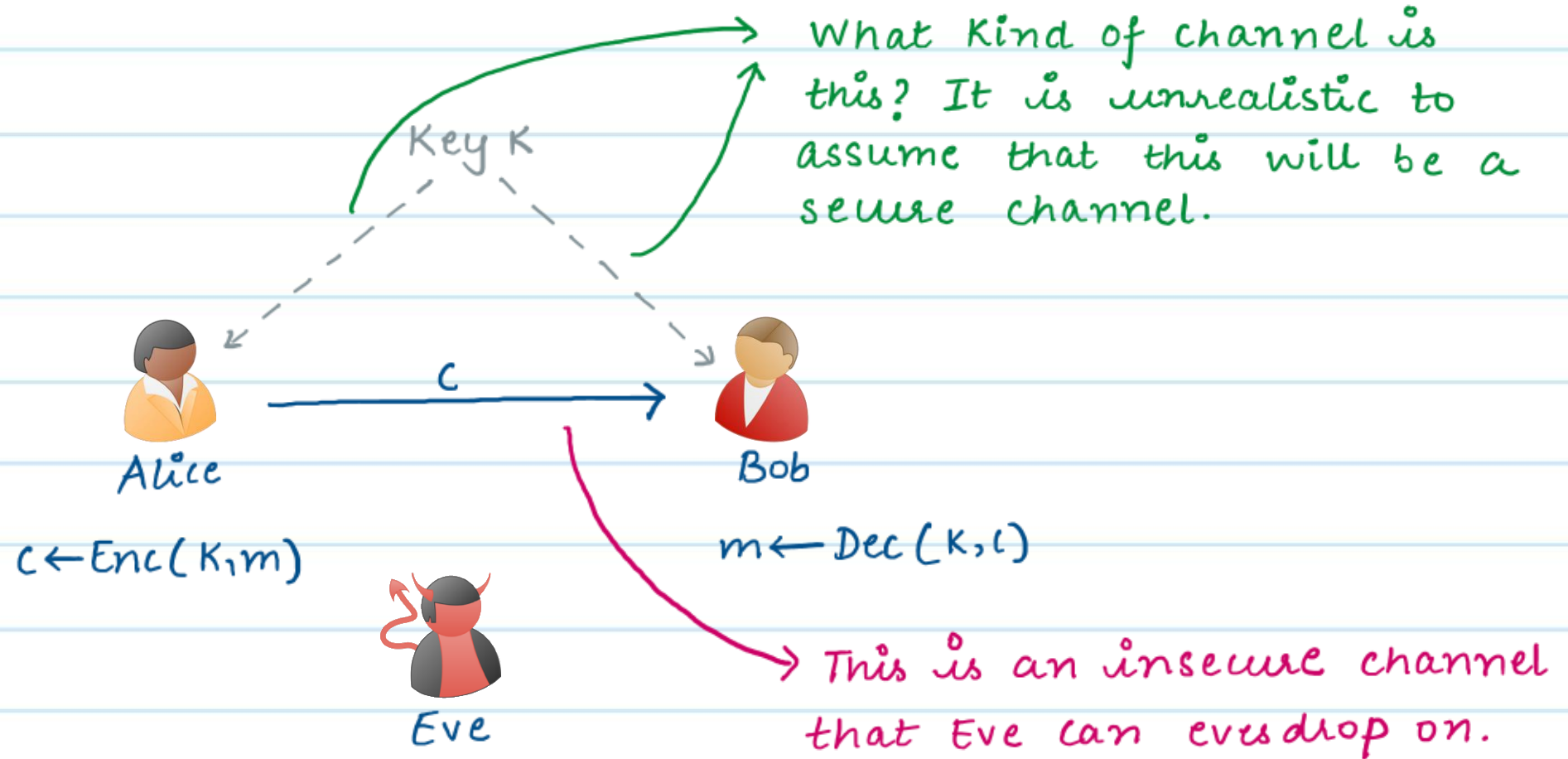
How Good Is This?

	Theoretical Construction	ECB	CBC	OFB	Ideal Goal CTR
Randomness Used	tn -bits	0 -bits	n -bits	n -bits	n -bits
Ciphertext Size	$2tn$ -bits	tn -bits	$tn+n$ -bits	$tn+n$ -bits	$tn+n$ -bits
Parallelizable ciphertext computation	yes	yes	No	No, but pre-computable	yes
Reusable Randomness	No	-	-	-	-
Minimal Assumption	PRF	PRP	PRP	PRF	PRF
CPA Secure?	Yes	No	yes	yes	yes

Recap (Key-Exchange)

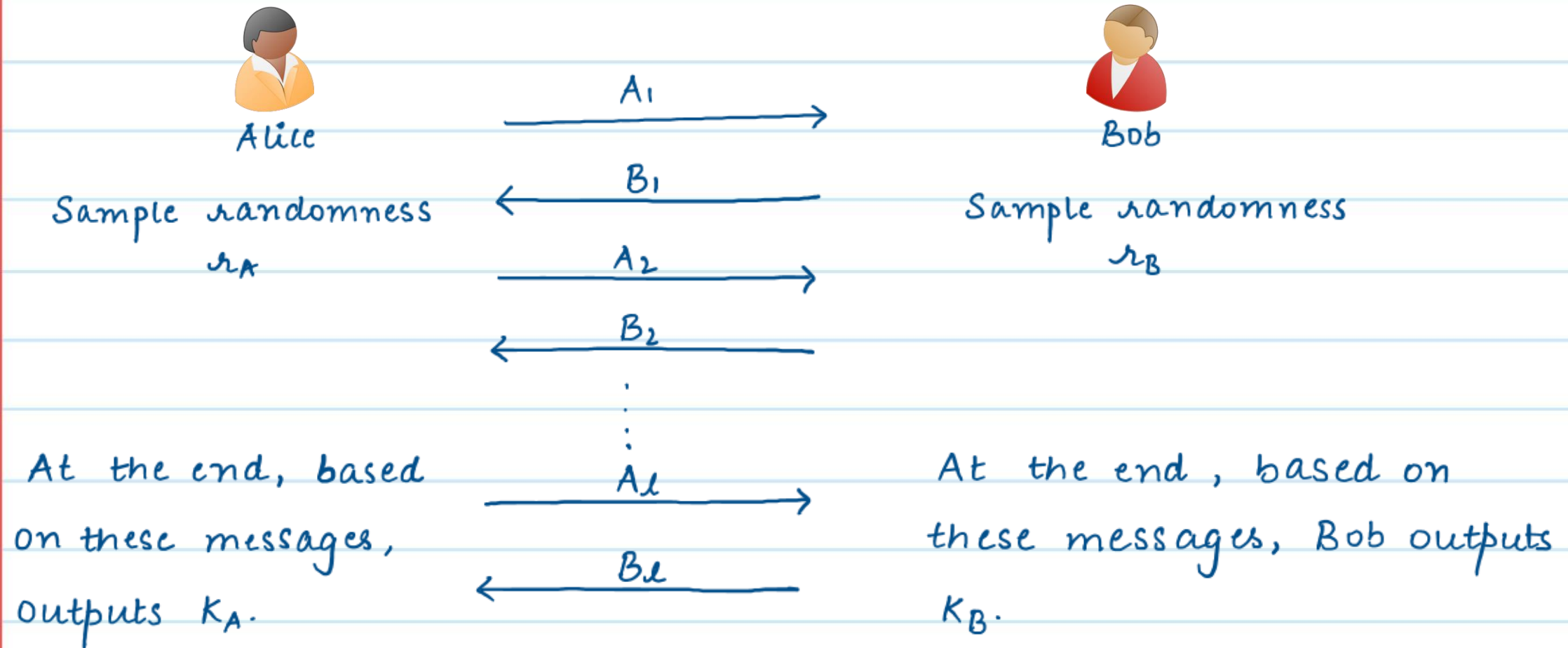
Key-Exchange Problem

* How do Alice and Bob share this key?



Key-Exchange Protocol

- * A way to establish a secret key via insecure communication channel.
- * A protocol is an **interactive algorithm**.



Diffie - Hellman Assumptions

Let's sample a cyclic group $(G, \cdot)$ of order q , with generator g .

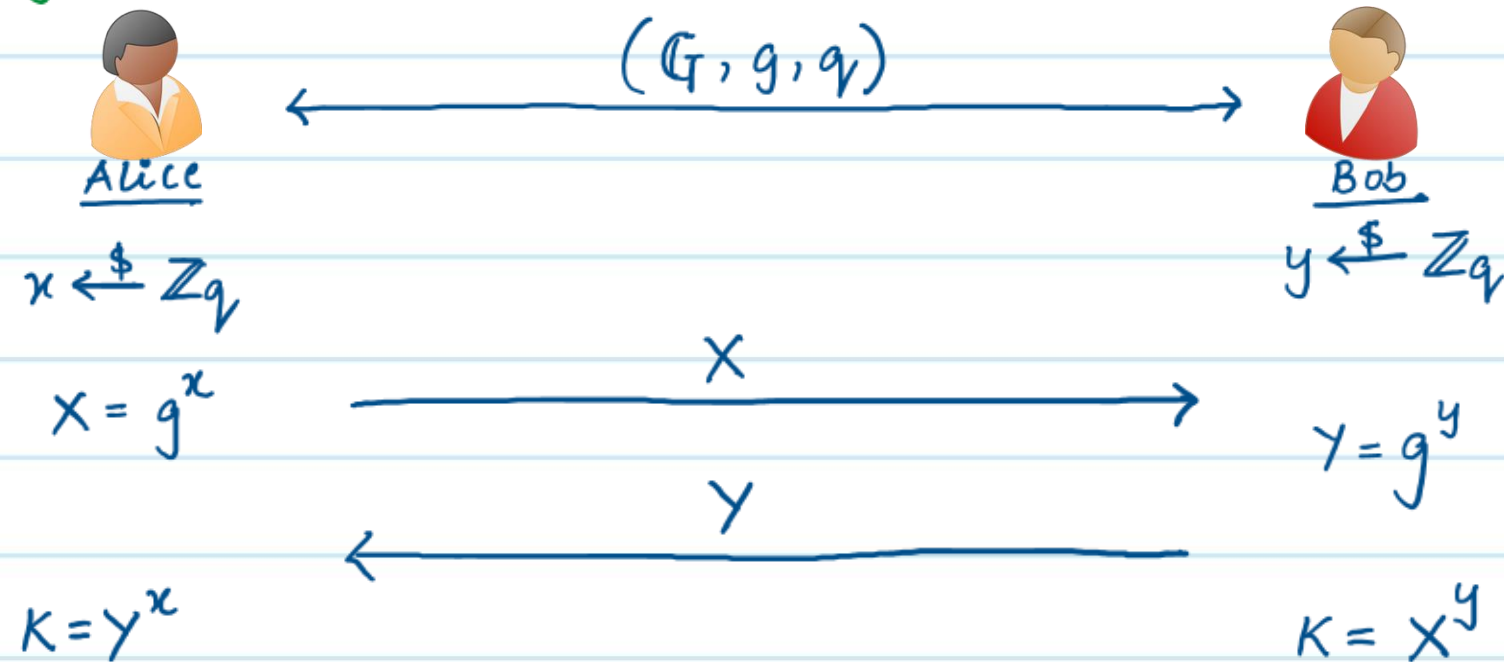
* Decisional Diffie-Hellman (DDH) Assumption: Sample $x, y, z \xleftarrow{\$} \mathbb{Z}_q$, compute $h_1 = g^x$, $h_2 = g^y$. Given (G, q, g, h_1, h_2) , it is computationally hard to distinguish between g^{xy} and g^z .

In other words,

$$(G, g, q, h_1, h_2, g^{xy}) \underset{\downarrow}{\approx}_c (G, g, q, h_1, h_2, g^z)$$

computationally
indistinguishable.

Diffie-Hellman Key-Exchange Protocol



Correctness. $Y^x = (g^y)^x = g^{xy}$
 $X^y = (g^x)^y = g^{xy}$

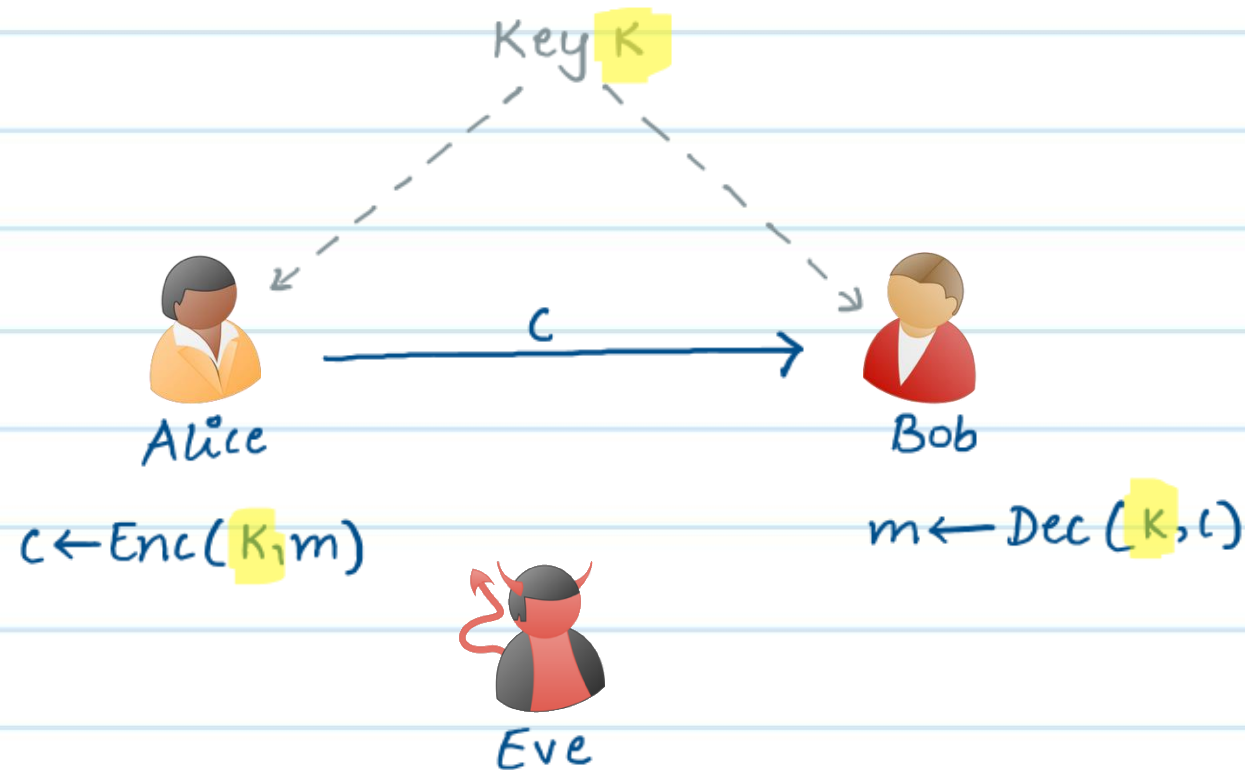
Both Alice and Bob
compute the same key.

Privacy: Eve only sees (G, g, q) ,
 $X = g^x$ & $Y = g^y$. From the DDH
assumption, we know that this
does not leak information about
the key $K = g^{xy}$.

Public-Key Encryption

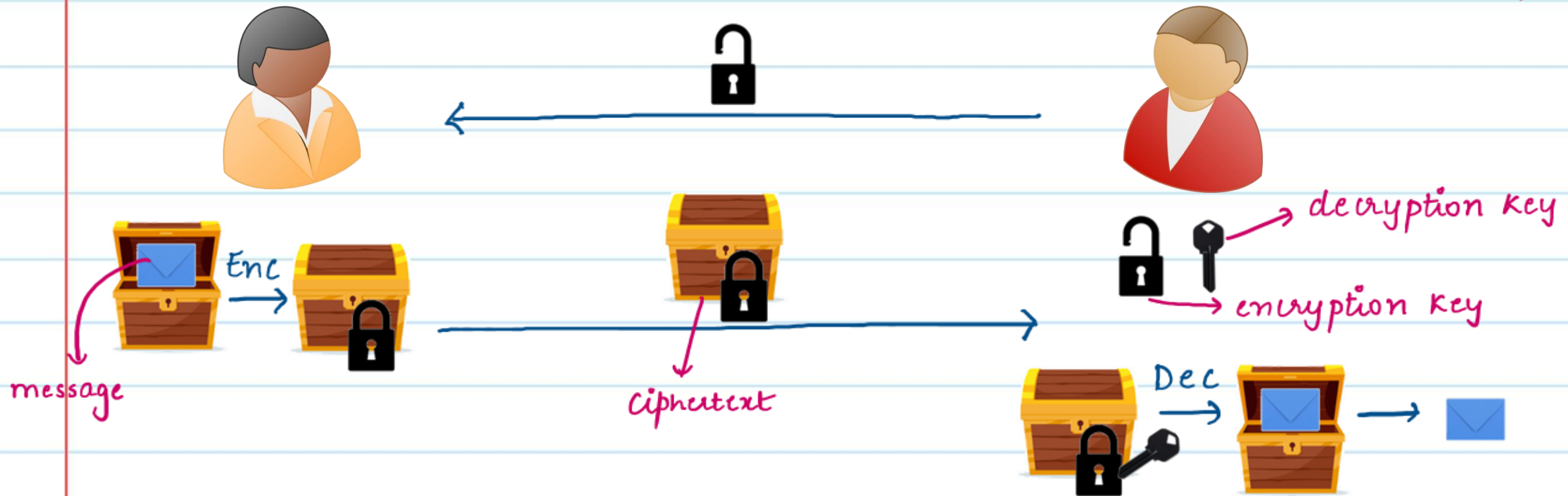
Secret-Key Encryption (SKE)

- * So far, when discussing encryption schemes, we assumed that Alice and Bob use the same *secret-key* for both encryption and decryption.
- * Such schemes are called secret-key encryption or symmetric-key encryption.



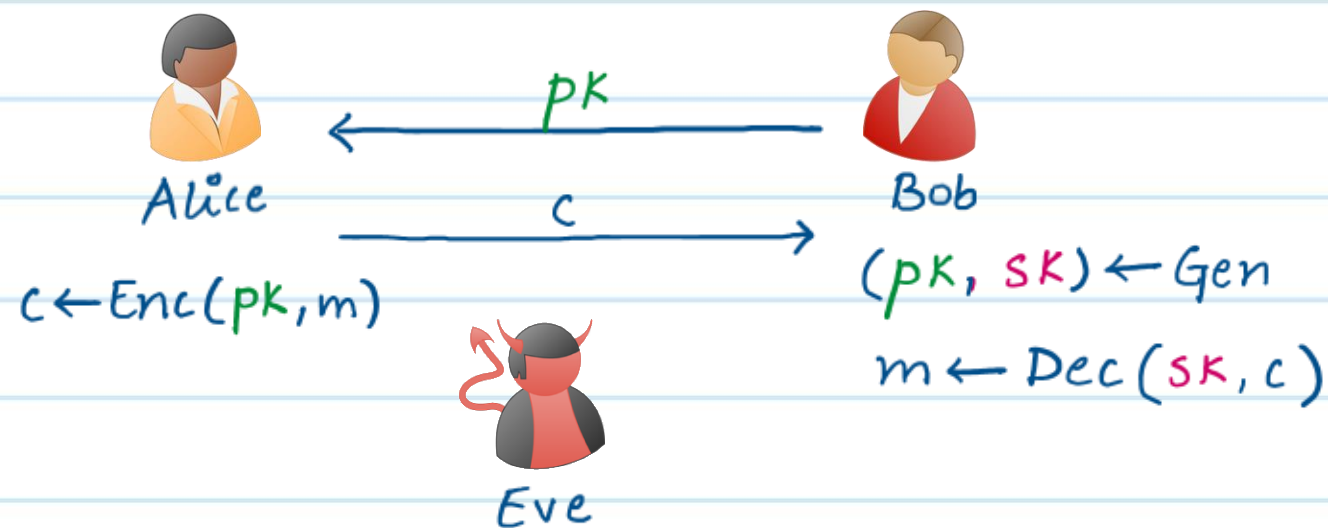
Public-Key Encryption (PKE)

- * Public-Key encryption is a different type of encryption, where the key used for encryption is different from the key used for decryption.



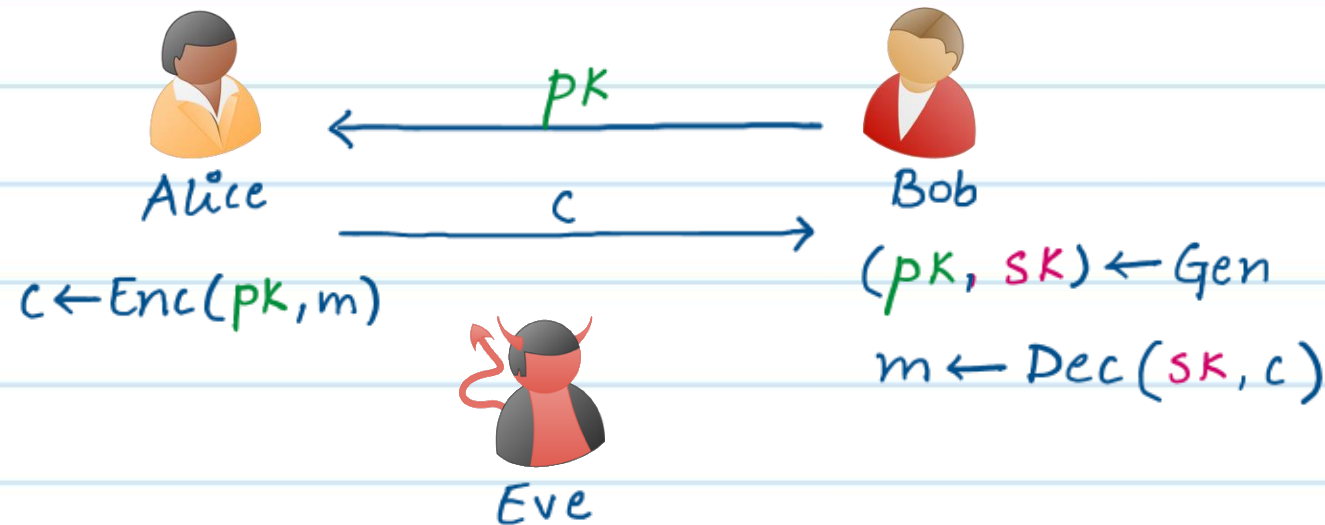
Public-Key Encryption

- * Encryption Key = public Key (pk)
- * Decryption Key = secret Key (sk)
- * If Bob wants to receive a private message from Alice, he can generate a (pk, sk) pair and send pk to Alice.
- * Alice uses pk to encrypt the message
- * Bob uses sk to decrypt the message



Public-Key Encryption

- * We don't need to hide the public key. It can be sent over a public communication channel. We only care about hiding the secret-key.
- * Public-Key does not reveal any information about the secret-key.
- * No need for a key-exchange protocol.



Public-key Encryption in the Real-world

*



End-to-end encrypted messaging

*



HTTPS (hypertext transfer protocol secure)

*



VPN (Virtual Private Network)

ElGamal Encryption

Public-key encryption for encrypting $m \in \mathbb{G}$.

- * **Gen**: Sample a cyclic group $(\mathbb{G}, \cdot)$ of order q with generator g .
Sample $x \xleftarrow{\$} \mathbb{Z}_q$. $pk = (\mathbb{G}, g, q, X = g^x)$, $sk = (\mathbb{G}, g, q, x)$
- * **Enc**(pk, m): Sample $y \xleftarrow{\$} \mathbb{Z}_q$. $c = (Y = g^y, D = m \cdot X^y)$
- * **Dec**(sk, c): Parse $c = (Y, D)$. Compute $m = D \cdot Y^{-x}$.

* Correctness: We need to show $\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m] = 1$.

$$\text{Enc}(\text{pk}, m) = (Y = g^y, D = m \cdot X^y)$$

$$\text{Dec}(\text{sk}, c): D \cdot Y^{-x} = m \cdot X^y \cdot Y^{-x} = m \cdot g^{xy} \cdot g^{-xy} = m$$

* Security: We need to show:

$$\forall m_0, m_1 \in \mathcal{M}: \{ \text{pk}, \underset{\downarrow}{\text{Enc}(\text{pk}, m_0)} \} \approx_c \{ \text{pk}, \underset{\downarrow}{\text{Enc}(\text{pk}, m_1)} \}$$

$$\{ G, g, q, g^x, g^y, m_0 \cdot g^{xy} \} \quad \{ G, g, q, g^x, g^y, m_1 \cdot g^{xy} \}$$

$$\{G, g, q, g^x, g^y, m_0 \cdot g^{xy}\}$$

$$\approx_c \{G, g, q, g^x, g^y, m_0 \cdot g^z\}$$

random
 $z \leftarrow \mathbb{Z}_q$

follows from the DDH assumption

$$\equiv \{G, g, q, g^x, g^y, m_1 \cdot g^z\}$$

perfectly indistinguishable
 (like one-time pad)

$$\approx_c \{G, g, q, g^x, g^y, m_1 \cdot g^{xy}\}$$

follows from the DDH assumption